INTRODUCTION TO INFO. & COMM. TECHNOLOGY

12-10-2023

Binary Multiplication

The Binary multiplication obeys the four basic rules:

0 x 0 = 0

1 x 0 = 0

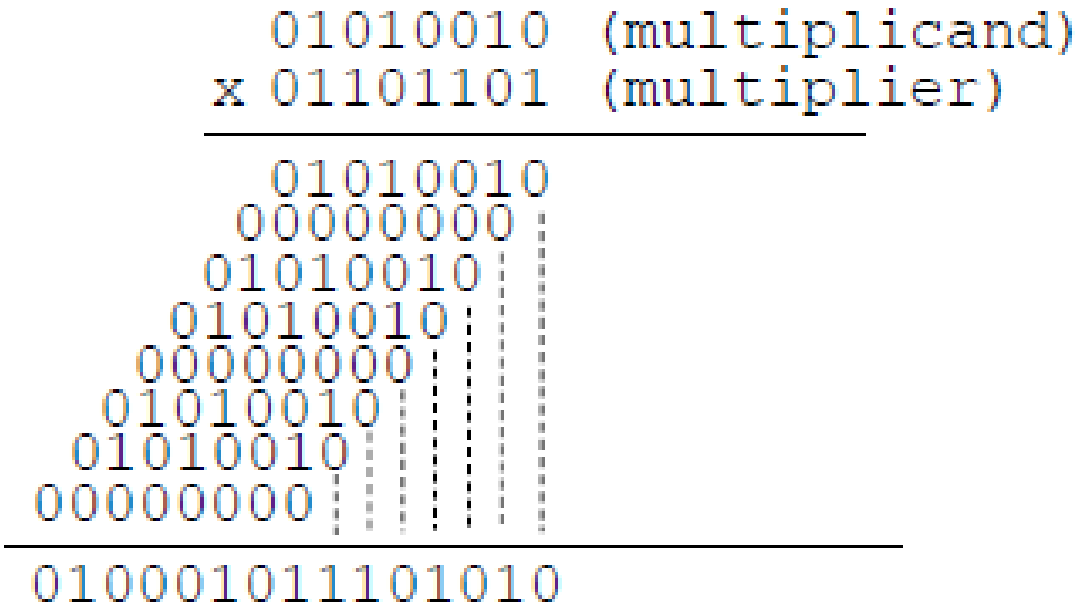
0 x 1 = 0

1 x 1 = 1

Binary Multiplication

```
    1000
x   1001
-----------
    1000
   0000
  0000
 1000
-----------
 1001000
```

Binary Multiplication



Binary Multiplication

1. 10 x10
2. 100 x 11
3. 101 x 10
4. 1011 x11
5. 11011 x 101

Answers

1. 100
2. 1100
3. 1010
4. 10 0001
5. 1000 0111

Binary Division

```
                 1001        Quotient
1000    |     1001010        Dividend
            -1000
                 10
                 101
                 1010
                -1000
                   10        Remainder
```

Verifying division

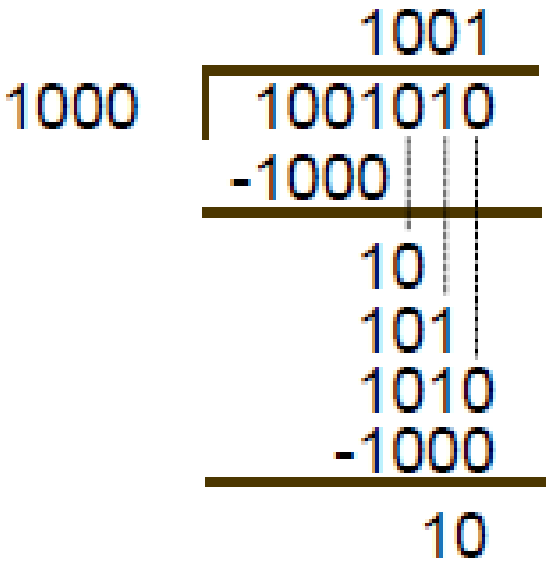
Dividend = Quotient x Divisor + Remainder

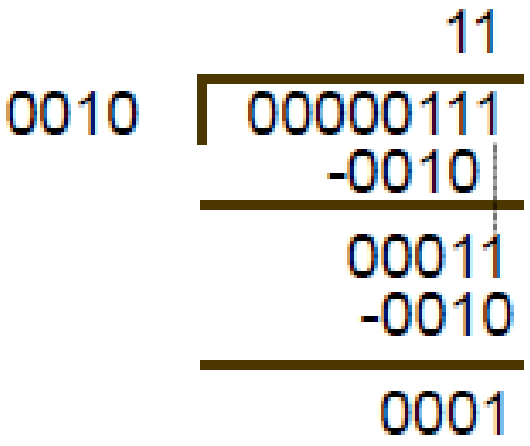
1001010 = (1001 x 1000) + 10

= 1001000 + 10

= 1001010

Division


1001
1000
1001010
-1000
10
101
1010
-1000
10


11
0010
00000111
-0010
00011
-0010
0001

Binary Division

1. 100 /10
2. 111/11
3. 1010 / 100
4. 1101 / 11
5. 10111 / 10

Answers

1. 10
2. 10 (R = 0001)
3. 10 (R = 0010)
4. 100 (R = 0001)
5. 1011 (R = 0001)

More on Binary Arithmetic

- To be covered in Labs

Unsigned Integers

- Memory Locations → 16-64 bits in length
- Unsigned form of an integer
- For each decimal number we enter, it is converted into binary and stored
- It must match the number of bits it takes up in the computer's memory
- You need to add 0s to the left of the number if the converted binary does not match the bits used by the computer for integer representation

Unsigned integers in Computers

- Both 10_2 and 101101_2 must have the same length as of a storage location
- Leading zeros help us here to fill memory spaces
- Examples:
  - Store the decimal integer 6_{10} in a 4-bit memory location
  - Store the decimal integer 5_{10} in an 8-bit memory location
  - Store the decimal integer 928_{10} in a 16-bit memory location

Answers

i. 0110_2
ii. 00000101_2
iii. 0000011101000000_2

Overflow

- Every computer can handle a limited sized values
- If you try to store an unsigned integer that is bigger than the maximum allowed → **overflow** occurs
- E.g. 23_{10} in a 4-bit memory location cannot be stored as it is equal to : $(10111)_2$ → Overflow

Range of unsigned integers

- Unsigned integers are the easiest to convert from decimal
- take up the least amount of room in the computer's memory
- However, they do not allow for much flexibility
- Limited in how many numbers they can represented

Range Contd.

Table 2.6 Sample ranges of unsigned integers

Number of Bits	Range
8	0...255
16	0...65,535
32	0...4,294,967,295
64	0...18,446,740,000,000,000 approximately

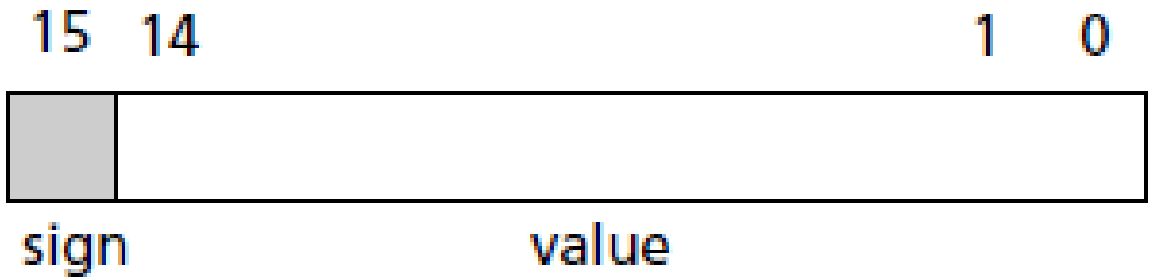
Two ways to represent Singed numbers

- Sign and magnitude
- Two’s complement

Sign-and-Magnitude Format

- To represent negative integers
- the leftmost bit is reserved to represent the sign
  - If the leftmost bit is 0, the number is positive
  - If the leftmost bit is 1, the number is negative
  - The other bits represent the magnitude (or the absolute value) of the integer

Sign-and-Magnitude Representation



Number = sign & value

- sign = 0 ⇒ +, positive number
- sign = 1 ⇒ -, negative number

Range of signed magnitude

Table 2.7 Sample ranges of sign-and-magnitude integers

Number of Bits	Range
8	-127...+127
16	-32,767...+32.767
32	-2,147,483,647...+2,147,483,647

Dec. No. → Sign-and-Magnitude Binary

i. $+23_{10}$ in an 8-bit memory location
ii. -19_{10} in an 8-bit memory location

- Answers :

i. 00010111 (binary of +23 = 00010111)

ii. 10010011 (binary of +19 = 00010011)

You can also convert it the other way round (signed binary to decimal)

Sign-and Magnitude Binary No → Dec.

- 00110111_2 is an 8-bit binary integer in sign-and-magnitude format, what is its decimal equivalent?
- Given that 10001110_2 is an 8-bit binary integer in sign-and-magnitude format, what is its decimal equivalent?

- Answers:

i. $+55_{10}$ (converted from 110111_2)

ii. -14_{10} (converted from 1110_2)

Problems with Sign Magnitude

- If we have 3 bits

= $-2^2+1 - 2^2-1$

= -3 … 3

Sign	Value	Number
0	0 0	0
0	0 1	1
0	1 0	2
0	1 1	3
1	0 0	?
1	0 1	- 1
1	1 0	- 2
1	1 1	- 3

Representing Zero

- Store the decimal integer 0 in an 8-bit memory location using sign-and-magnitude format:
- given that 10000000_2 is an 8-bit binary integer in sign-and-magnitude form, find its decimal value:
- both 00000000_2 and 10000000_2 to represent the same number → ??????????????
- The fact that 0 has two possible representations in sign-and-magnitude format → one of the main reasons why computers usually use a different method to represent signed integers

Problems with sign magnitude

- Given 2 binary numbers: one positive, one negative

	0 0 1 1 0 0 (base 2)	12	(base 10)
+	1 0 1 1 1 1	-15	
	1 1 1 0 1 1 (= -27_{10})	-3	

11011_2 = 27 → -ve due to leading 1

One's Complement Format

- Not often used
- Required when we are implementing 2's complement
- Change each 1 to a zero and each zero to a 1
- positive integers are represented as they would be in sign-and magnitude format
- The leftmost bit is still reserved as the sign bit
  - $+6_{10}$, in a 4-bit allocation, is still 0110_2.
  - -6_{10} is just the complement of $+6_{10}$.
  - So -6_{10} becomes 1001_2

1’s Complement method

- Store the decimal integer $+78_{10}$ in an 8-bit memory location using one’s complement format
- 78 to binary: 1001110
- 7 bits are used for storing the magnitude
- number is positive so add a zero in the leftmost place to show the positive sign
- $+78_{10}$ in one’s complement in an 8-bit location is 01001110

1's Complement

- Store the decimal integer −37 in an 8-bit memory location using one's complement format:
- 37 in binary: 100101
- 7 bits are used for storing the magnitude of the number
- The number 100101 uses 6 bits so add one 0 to the left to make up 7 bits: 0100101
- This number is negative
- Complement all the digits by changing all the 0s to 1s and all the 1s to 0s → 1011010
- Add a 1 in the 8th bit location to represent negative sign
- −37 → 11011010_2

Representing Zero now….

- 0000000_2
- 11111111_2
- Read why there is so much fuss about Zero (Pg. 108 and Pg. 111)

2's Complement – solving the zero issue

- If the number is positive
  - just convert the decimal integer to binary
- If the number is negative,
  - convert the number to binary and find the one's complement
  - Add a binary 1 to the one's complement of the number
  - If this results in an extra bit (more than X bits), discard the leftmost bit
- *A negative number is represented as the "2's Complement" of its binary*
  - One's complement + 1
  - or copy till first 1 then reverse bits (Right to Left)

2's Complement

- Find the two's complement of $+2_{10}$ as a 4-bit binary integer
- Convert 2 to binary: 10
- Add zeros to the left to complete 4 bits: 0010
- Since this is already a positive integer, you are done

2's Complement

- Find the two's complement of -2_{10} as a 4-bit binary integer:
- 2 in binary: 10
- Add zeros to the left to complete 4 bits: 0010
- Since the number is negative→ 1's complement = 1101
- Now add binary 1 to this number: 1110
- Therefore, −2 in two's complement in a 4-bit location is 1110

2’s Complement

- −43 as an 8-bit binary integer
- 43 in binary: 101011
- 8 bits code : 00101011
- Since the number is negative, 1’s complement = 11010100
- Add binary 1 to this number:
- 11010100 + 1 → 11010101

Program-1

- Write program logic for the following:
- Ask the user to enter three integer values
- Store them in x, y, and z
- Calculate the product of three values and store in PROD
- Calculate the square of each value and store in Sq_x, Sq_y, Sq_z
- Also calculate the sum of three entered values and store in SUM
- Display PROD, Sq_x, Sq_y, Sq_z, SUM
- Display a message at the end " That's all for now.."

Book Reference

- Chapter-5
- Book-2